

Review of digital logic design

Background

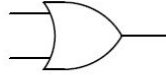
- Logic gates (AND, OR, NOT, XOR, etc.)
- Boolean algebra
- Truth tables
- Logic functions
- Logic function synthesis by
 - Sum of Products (SOP)
 - Product of Sums (POS)
 - K-maps
- Logic blocks (MUX, DEMUX)
- Sequential elements (Latch, Flip-flop)

Logic gates



AND

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1



OR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1



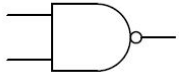
XOR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0



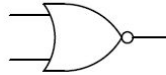
NOT

Input	Output
0	1
1	0



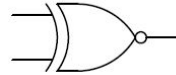
NAND

A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0



NOR

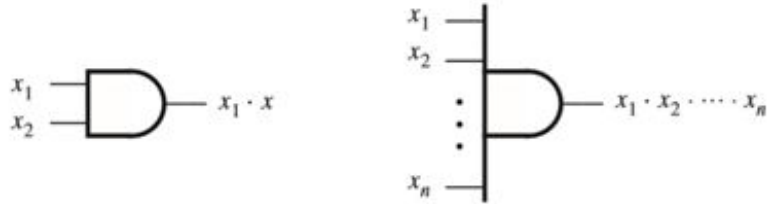
A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0



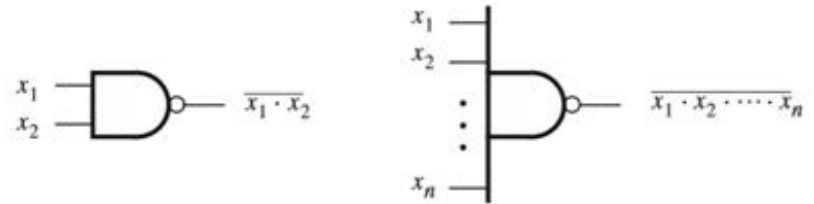
XNOR

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	1

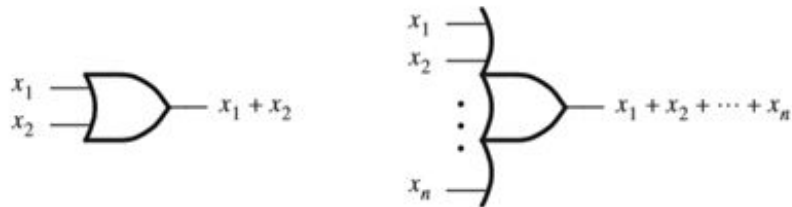
Generalized n-input logic gates



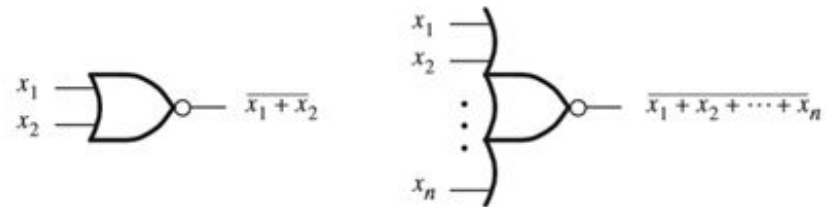
AND gates



NAND gates



OR gates



NOR gates

Axioms of Boolean Algebra

- 1a. $0 \cdot 0 = 0$
- 1b. $1 + 1 = 1$
- 2a. $1 \cdot 1 = 1$
- 2b. $0 + 0 = 0$
- 3a. $0 \cdot 1 = 1 \cdot 0 = 0$
- 3b. $1 + 0 = 0 + 1 = 1$
- 4a. If $x = 0$, then $x' = 1$
- 4b. If $x = 1$, then $x' = 0$

Boolean Algebra - Single Variable Theorems

- 5a. $x \cdot 0 = 0$
- 5b. $x + 1 = 1$
- 6a. $x \cdot 1 = x$
- 6b. $x + 0 = x$
- 7a. $x \cdot x = x$
- 7b. $x + x = x$
- 8a. $x \cdot x' = 0$
- 8b. $x + x' = 1$
- 9. $(x')' = x$

● Boolean Algebra - Two Variable Properties

- 10a. $x \cdot y = y \cdot x$ Commutative
- 10b. $x + y = y + x$
- 11a. $x \cdot (y \cdot z) = (x \cdot y) \cdot z$ Associative
- 11b. $x + (y + z) = (x + y) + z$
- 12a. $x \cdot (y + z) = x \cdot y + x \cdot z$ Distributive
- 12b. $x + y \cdot z = (x + y) \cdot (x + z)$
- 13a. $x + x \cdot y = x$ Absorption
- 13b. $x \cdot (x + y) = x$
- 14a. $x \cdot y + x \cdot y' = x$ Combining
- 14b. $(x + y) \cdot (x + y') = x$

Boolean Algebra - Two & Three Variable Properties

- 15a. $(x \cdot y)' = x' + y'$
- 15b. $(x + y)' = x' \cdot y'$
- 16a. $x + x' \cdot y = x + y$
- 16b. $x \cdot (x' + y) = x \cdot y$
- 17a. $x \cdot y + y \cdot z + x' \cdot z = x \cdot y + x' \cdot z$ Consensus
- 17b. $(x + y) \cdot (y + z) \cdot (x' + z) = (x + y) \cdot (x' + z)$

DeMorgan's theorem

Logic Function Synthesis - Three variable SOP & POS

- Function synthesis from truth table

Row number	x_1	x_2	x_3		Minterm	Maxterm
0	0	0	0		$m_0 = \bar{x}_1\bar{x}_2\bar{x}_3$	$M_0 = x_1 + x_2 + x_3$
1	0	0	1		$m_1 = \bar{x}_1\bar{x}_2x_3$	$M_1 = x_1 + x_2 + \bar{x}_3$
2	0	1	0		$m_2 = \bar{x}_1x_2\bar{x}_3$	$M_2 = x_1 + \bar{x}_2 + x_3$
3	0	1	1		$m_3 = \bar{x}_1x_2x_3$	$M_3 = x_1 + \bar{x}_2 + \bar{x}_3$
4	1	0	0		$m_4 = x_1\bar{x}_2\bar{x}_3$	$M_4 = \bar{x}_1 + x_2 + x_3$
5	1	0	1		$m_5 = x_1\bar{x}_2x_3$	$M_5 = \bar{x}_1 + x_2 + \bar{x}_3$
6	1	1	0		$m_6 = x_1x_2\bar{x}_3$	$M_6 = \bar{x}_1 + \bar{x}_2 + x_3$
7	1	1	1		$m_7 = x_1x_2x_3$	$M_7 = \bar{x}_1 + \bar{x}_2 + \bar{x}_3$

- A **Minterm** m_i is a complete argument vector **(a,b,c,...,x)** for which a Boolean function $f(a,b,c,...,x)$ delivers the value '1'. It has "**minimum**" **satisfiability**.
- A **Maxterm** M_i is a complete argument vector **(a,b,c,...,x)** for which a Boolean function $f(a,b,c,...,x)$ delivers the value '0'. It has "**maximum**" **satisfiability**.

SOP: Sum of product

- $ABC' + B'CD + ABCD + AD'$
- $A \square 1, A' \square 0$

POS: Product of sum

- $(A+B+C')(A+D')(B'+C+D)$
- $A \square 0, A' \square 1$

Logic Function Synthesis - Three variable SOP & POS

Example: 01: Write down the Boolean expression in **SOP** form that describes this truth table

A	B	C	Q
0	0	0	0
0	1	0	1
1	0	0	1
1	1	0	0
0	0	1	1
0	1	1	0
1	0	1	0
1	1	1	1

$$\bar{A}.B.\bar{C} + A.\bar{B}.\bar{C} + \bar{A}.\bar{B}.C + A.B.C = Q$$

Example: 02: Write down the Boolean expression in **POS** form that describes this truth table

Sensor Inputs			
A	B	C	Output
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Logic Function Synthesis - Three variable SOP & POS

Example: 01: Write down the Boolean expression in **POS** form that describes this truth table

A	B	C	Output	Output	
0	0	0	0	0	$(A + B + C)$
0	0	1	0	1	
0	1	0	0	1	
0	1	1	1	1	
1	0	0	0	1	$(\bar{A} + \bar{B} + \bar{C})$
1	0	1	1	1	
1	1	0	1	1	
1	1	1	1	0	

$$\text{Output} = (A + B + C) (\bar{A} + \bar{B} + \bar{C})$$

Logic Function Synthesis - 2/3/4 variable k-map

- Function synthesis using k-maps

x_1	x_2	
0	0	m_0
0	1	m_1
1	0	m_2
1	1	m_3

x_1	x_2	
0	0	m_0
0	1	m_2
1	0	m_1
1	1	m_3

x_1	x_2	x_3	
0	0	0	m_0
0	0	1	m_1
0	1	0	m_2
0	1	1	m_3
1	0	0	m_4
1	0	1	m_5
1	1	0	m_6
1	1	1	m_7

$x_1 x_2$	x_3	00	01	11	10
0		m_0	m_2	m_6	m_4
1		m_1	m_3	m_7	m_5

$x_1 x_2$	$x_3 x_4$	00	01	11	10
00		m_0	m_4	m_{12}	m_8
01		m_1	m_5	m_{13}	m_9
11		m_3	m_7	m_{15}	m_{11}
10		m_2	m_6	m_{14}	m_{10}

Logic Function Synthesis - 2/3/4 variable k-map

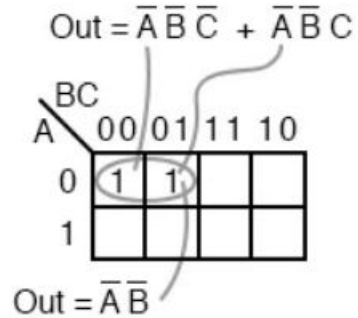
Function synthesis using **SOP** in k-maps

1. No zeros allowed.
2. No diagonals.
3. Only **power of 2** number of cells in each group. ($2^0=1$, $2^1=2$, $2^2=4$, $2^3=8$, etc.)
4. Groups should be as large as possible.
5. Every 1 must be in at least one group.
6. **Overlapping allowed.**
7. **Wrap around allowed.**
8. **Fewest number of groups** possible.

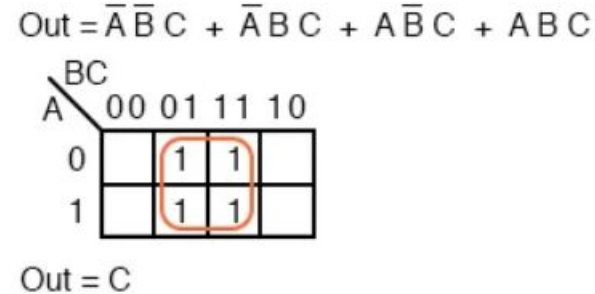
Visit: <http://www.ee.surrey.ac.uk/Projects/Labview/minimisation/karrules.html>

3 variable K-Map

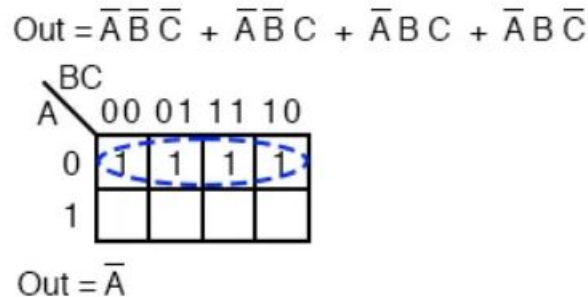
Example: 01



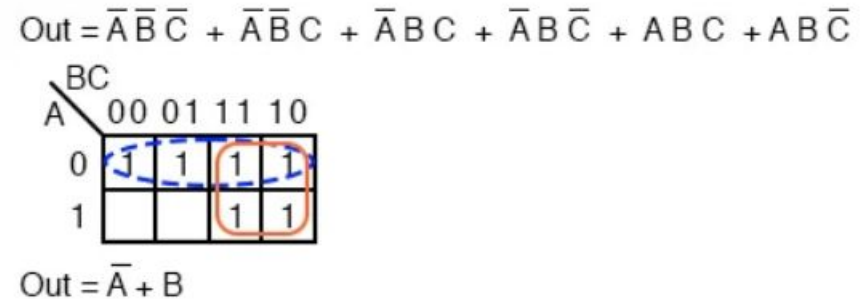
Example: 03



Example: 02



Example: 04



3 variable K-Map

Example: 05

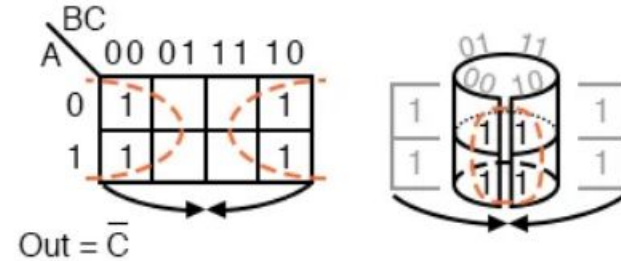
$$\text{Out} = \bar{A} B C + A B C$$

BC A	00	01	11	10
	0	1	1	0
0			1	
1			1	

$$\text{Out} = B C$$

Example: 06

$$\text{Out} = \bar{A} \bar{B} \bar{C} + A \bar{B} \bar{C} + \bar{A} B \bar{C} + A B \bar{C}$$



Example: 07

$$\text{Out} = \bar{A} \bar{B} \bar{C} + \bar{A} \bar{B} C + \bar{A} B C + \bar{A} B \bar{C} + A \bar{B} \bar{C} + A B \bar{C}$$

BC A	00	01	11	10
	0	1	1	0
0	1	1	1	1
1	1			1

$$\text{Out} = \bar{A} + \bar{C}$$

Example: 08

BC A	00	01	11	10
	0	1	1	0
0			1	
1		1	1	1

$$\text{Output} = AB + BC + AC$$

4 variable K-Map

Example: 01

$$\text{Out} = \bar{A}\bar{B}CD + \bar{A}BCD + ABCD + A\bar{B}CD + AB\bar{C}\bar{D} + AB\bar{C}D + ABC\bar{D}$$

		CD			
		00	01	11	10
A B	00			1	
	01			1	
	11	1	1	1	1
	10			1	

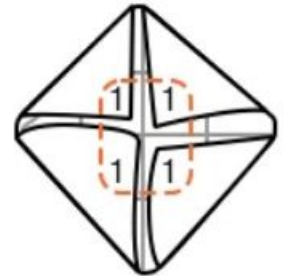
$$\text{Out} = AB + CD$$

Example: 02

$$\text{Out} = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} + A\bar{B}\bar{C}\bar{D} + A\bar{B}C\bar{D}$$

		CD			
		00	01	11	10
A B	00	1			1
	01				
	11				
	10	1			1

$$\text{Out} = \bar{B}\bar{D}$$



4 variable K-Map

Example: 03

$$\text{Out} = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}CD \\ + A\overline{B}\overline{C}\overline{D} + A\overline{B}\overline{C}D + A\overline{B}C\overline{D} + A\overline{B}CD$$

A \ B \ CD				
	00	01	11	10
00	1	1	1	1
01				
11				
10	1	1	1	1

$$\text{Out} = \overline{B}$$

Example: 04

$$\text{Out} = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}CD \\ + B\overline{C}\overline{D} + B\overline{C}D + A\overline{B}\overline{C}\overline{D} + A\overline{B}D + A\overline{B}C\overline{D}$$

A \ B \ CD				
	00	01	11	10
00	1	1	1	1
01	1			1
11	1			1
10	1	1	1	1

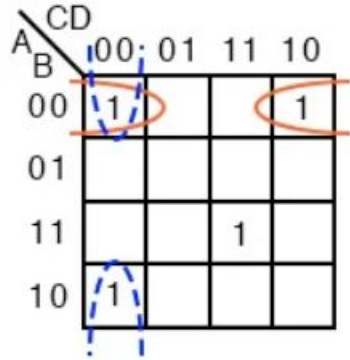
A \ B \ CD				
	00	01	11	10
00	1	1	1	1
01	1			1
11	1			1
10	1	1	1	1

$$\text{Out} = \overline{B} + \overline{D}$$

4 variable K-Map

Example: 05

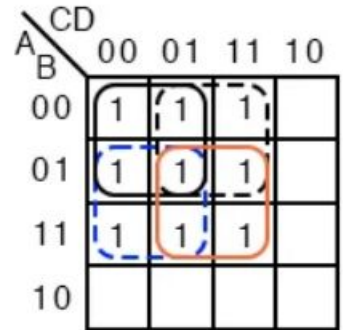
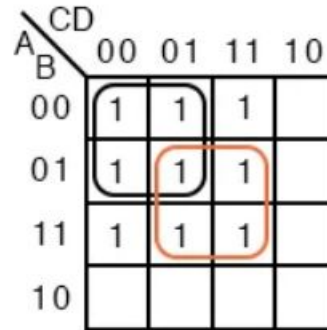
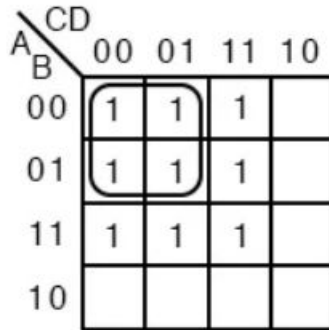
$$\text{Out} = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}C\overline{D} + A\overline{B}\overline{C}\overline{D} + ABCD$$



$$\text{Out} = \overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{D} + ABCD$$

Example: 06

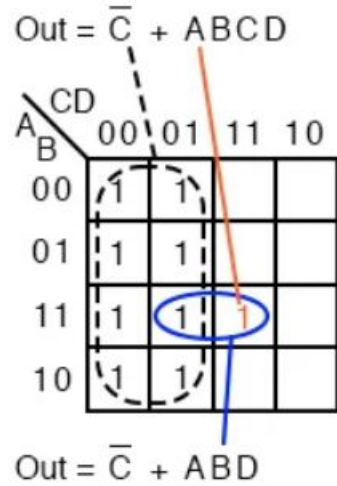
$$\begin{aligned}\text{Out} = & \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} \\ & + \overline{A}B\overline{C}\overline{D} + \overline{A}B\overline{C}D + \overline{A}BC\overline{D} \\ & + AB\overline{C}\overline{D} + AB\overline{C}D + ABCD\end{aligned}$$



$$\text{Out} = \overline{A}\overline{C} + \overline{A}D + B\overline{C} + BD$$

4 variable K-Map

Example: 07



Simplification by Boolean Algebra

$$\text{Out} = \bar{C} + ABCD$$

Applying rule $A + \bar{A}B = A + B$ to the $\bar{C} + ABCD$ term

$$\text{Out} = \bar{C} + ABD$$

Don't Care Condition

Example: 01

L1

BC	00	01	11	10
A=0	0	0	1	1
A=1	1	1	*	*

$$L1 = A + B + C$$

L2

BC	00	01	11	10
A=0	0	0	1	1
A=1	1	1	*	*

$$L2 = A + B$$

L3

BC	00	01	11	10
A=0	0	0	1	1
A=1	1	1	*	*

$$L3 = A + BC$$

L4

BC	00	01	11	10
A=0	0	0	0	0
A=1	1	1	*	*

$$L4 = A$$

L5

BC	00	01	11	10
A=0	0	0	0	0
A=1	1	1	*	*

$$L5 = AC$$

*** Don't care cells may be used as either 1s or 0s, whichever is useful

There are four sensors, A, B, C, and D that control the lights in a room. Each sensor transitions to a **HIGH** state when it detects an ambient light level exceeding a predefined threshold. The room's lights will turn **ON** when the majority of sensors are in the **LOW** state. If the number of sensors in the **HIGH** and **LOW** states is equal, the room's lighting condition remains **indeterminate** (i.e., a don't care condition). Answer questions (a-c) below based on this description.

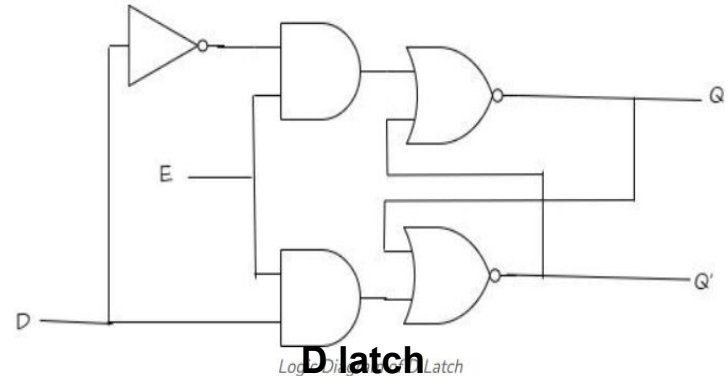
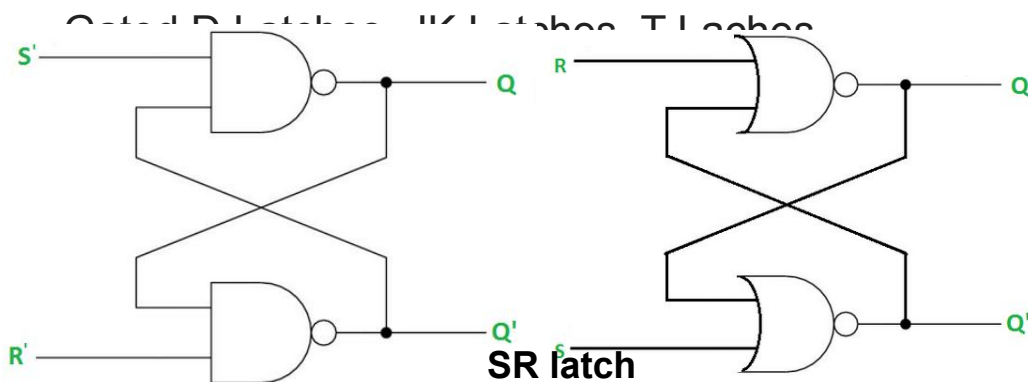
- (a) [2 marks] **Write** down the truth table for the system, describing all possible states of the sensors (A, B, C, D) and their corresponding output states.
- (b) [5 marks] **Draw** the Karnaugh map (K-map) for the system based on the truth table in part (a). **Derive** the logical expression that governs the system from the K-map.
- (c) [5 marks] **Draw** the CMOS logic circuit for the logical expression derived in part (b). **Specify** the number of MOSFETs required to implement your design.

A	B	C	D	Y	
0	0	0	0	0	→ OFF
0	0	0	1	0	→ OFF
0	0	1	0	0	→ OFF
0	0	1	1	d	→ don't care
0	1	0	0	0	→ OFF
0	1	0	1	d	→ don't care
0	1	1	0	d	→ don't care
0	1	1	1	1	→ ON
1	0	0	0	0	→ OFF
1	0	0	1	d	→ don't care
1	0	1	0	d	→ don't care
1	0	1	1	1	→ ON
1	1	0	0	d	→ don't care
1	1	0	1	1	→ ON
1	1	1	0	1	→ ON
1	1	1	1	1	→ ON

	00	01	11	10
00	0	0	d	0
01	0	d	1	d
11	d	1	1	1
10	0	d	1	d

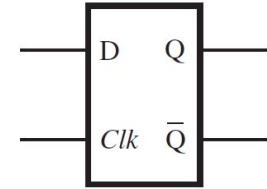
Latch

- Latches are digital circuits that can store a single bit of information and hold its value until it is updated by new input signals. Latches are **asynchronous**. (Unlike flip-flops that update only on the edge of a clock (rising or falling), latches update immediately when the enabling level is maintained.)
- They are used in digital systems as temporary storage elements to store binary information.
- Latches can be implemented using various digital logic gates, such as **AND, OR, NOT, NAND, and NOR** gates.
- Different types of latches: SR (Set-Reset) Latches, Gated SR Latches, D Latches,

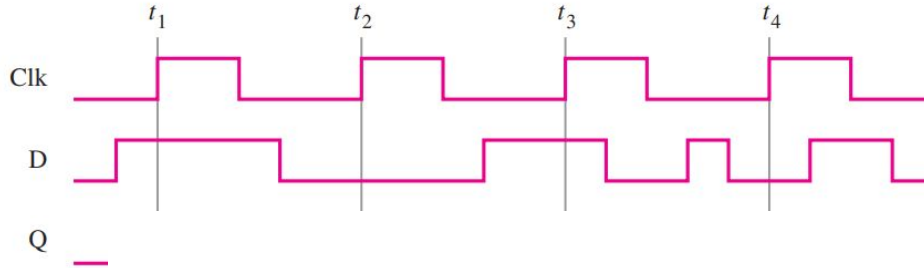


D Latch

- Level sensitive element
- *A positive level triggered D latch*
 - copies D to output Q, if Clock=1, else preserves the previous output
- *A negative level triggered D latch*
 - copies D to output Q, if Clock=0, else preserves the previous output



Graphical symbol

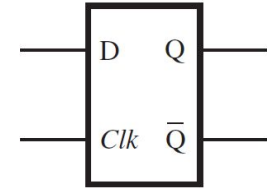


Clk	D	$Q(t+1)$
0	x	$Q(t)$
1	0	0
1	1	1

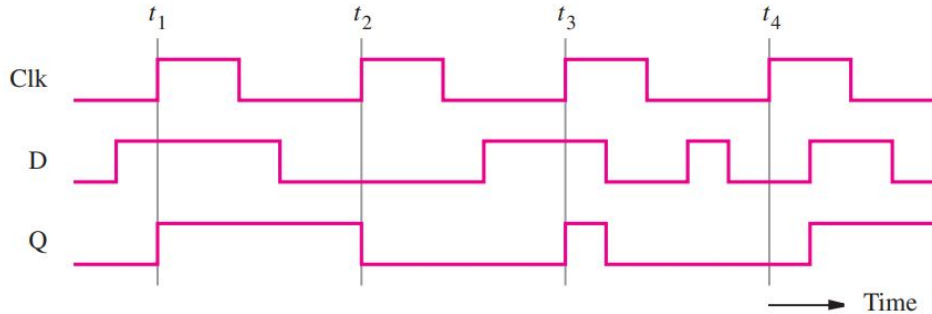
Characteristic table

D Latch

- Level sensitive element
- A **positive level triggered** D latch
 - copies D to output Q, if Clock=1, else preserves the previous output
- A **negative level triggered** D latch
 - copies D to output Q, if Clock=0, else preserves the previous output



Graphical symbol



Clk	D	Q(t + 1)
0	x	Q(t)
1	0	0
1	1	1

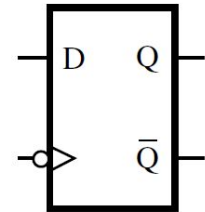
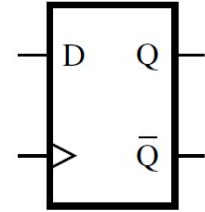
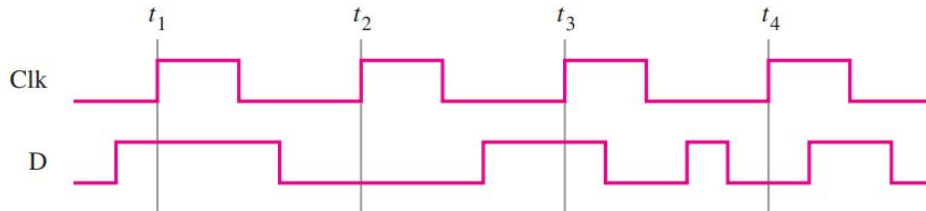
Characteristic table

D Flip-flop

- Edge sensitive element



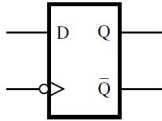
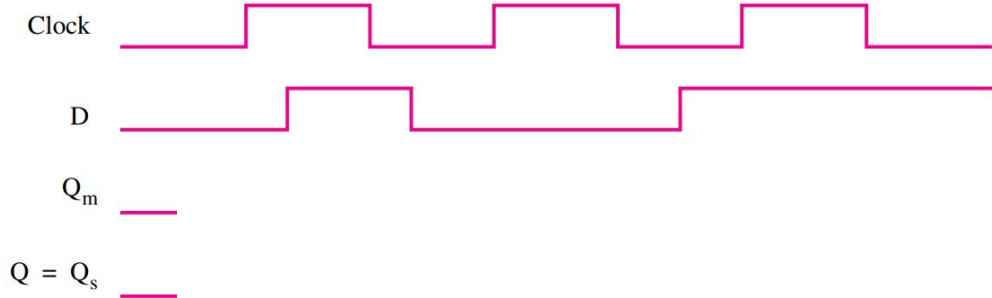
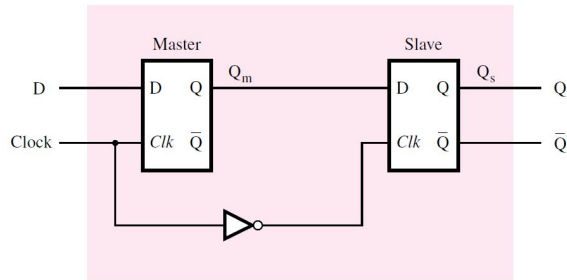
- A **positive edge triggered** D flip-flop
 - Sets $Q=D$ at all positive edges (rising edges) of the clock, retains the old value of Q otherwise
- A **negative edge triggered** D flip-flop
 - Sets $Q=D$ at all negative edges (falling edges) of the clock, retains the old value of Q otherwise



Graphical symbol

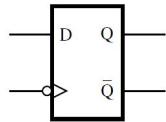
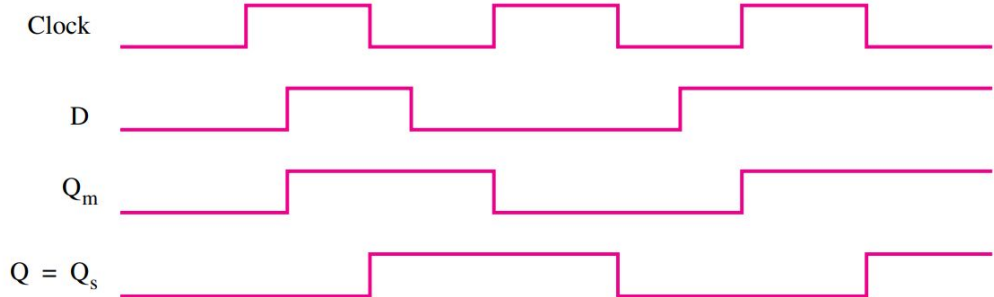
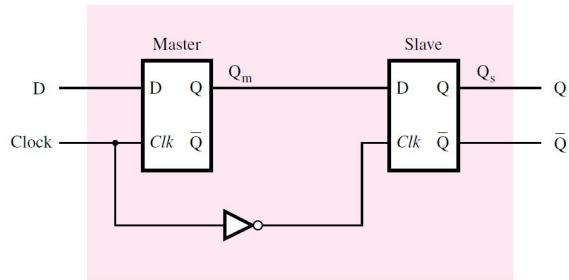
Building -ve D Flip-flop using D Latches

- By cascading a **positive** level triggered D latch and a **negative** level triggered D latch we can build a **negative** edge triggered D flip-flop



Building -ve D Flip-flop using D Latches

- By cascading a positive level triggered D latch and a negative level triggered D latch we can build a negative edge triggered D flip-flop

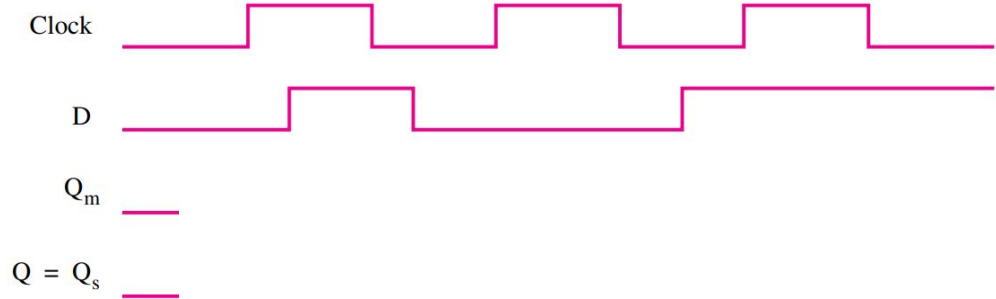
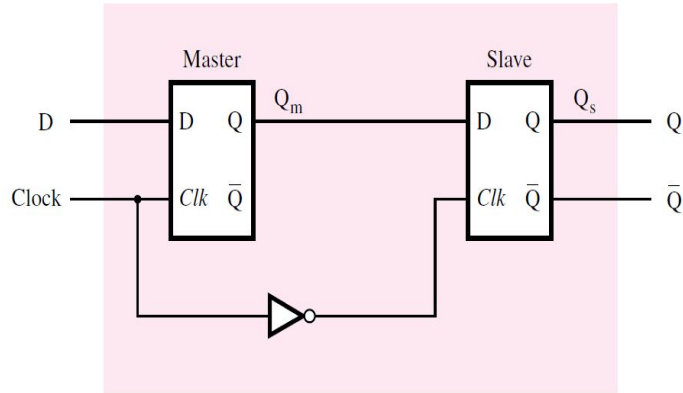


symbol of negative
edge Flipflop

follow the Q_m, clock_bar signals to draw Q_s ie, Q
of the second latch

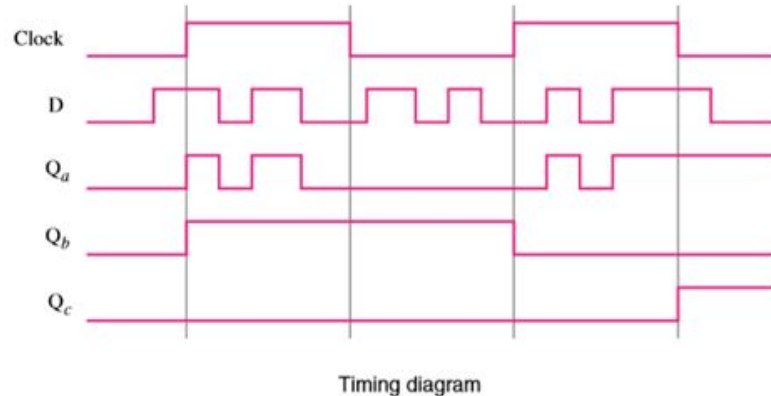
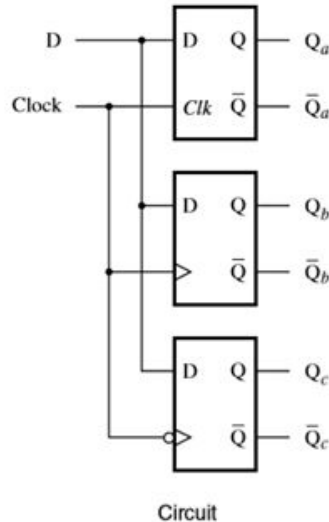
Building +ve D Flip-flop using D Latches

- By cascading a **negative** level triggered D latch and a **positive** level triggered D latch we can build a **Positive** edge triggered D flip-flop



Level triggered vs. Edge triggered

- In level triggered elements
 - output is affected by the clock levels (*high/low*)
- In edge triggered elements
 - output is affected by the clock edges (positive edge/negative edge) (rising edge/falling edge)



Level triggered vs. Edge triggered

- In level triggered elements
 - output is affected by the clock levels (*high/low*)
- In edge triggered elements
 - output is affected by the clock edges (positive edge/negative edge) (rising edge/falling edge)

